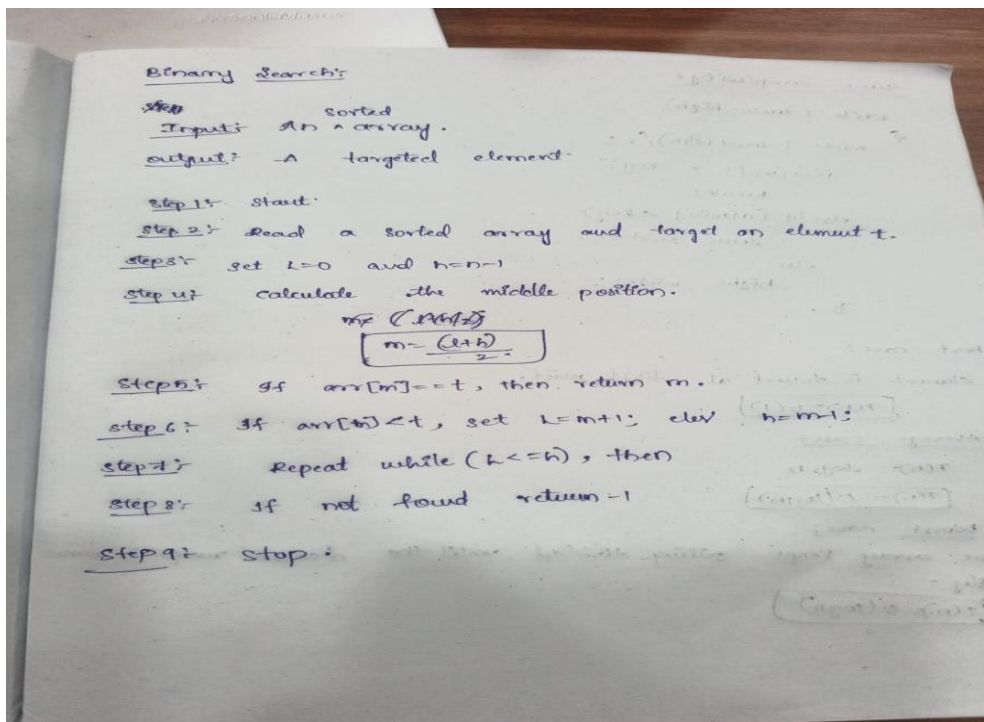


Practical -02

Aim : To Implementation and Time analysis of linear and binary search algorithms.

Algorithm:



Program :

```
#include <iostream>

using namespace std;

class LinearSearchApp
{
public:
    int arr[100], n;

    void arrayInput()
```

```
{  
    cout << "Enter number of elements: ";  
    cin >> n;  
    cout << "Enter the elements: ";  
    for (int i = 0; i < n; i++)  
    {  
        cin >> arr[i];  
    }  
}  
void printArray()  
{  
    for (int i = 0; i < n; i++)  
    {  
        cout << arr[i] << " ";  
    }  
    cout << endl;  
}  
// Linear Search Function  
int search(int target)  
{  
    for (int i = 0; i < n; i++)  
    {  
        if (arr[i] == target)  
        {  
            return i;  
        }  
    }  
}
```

```
    }  
    return -1;  
}  
};  
  
int main()  
{  
    LinearSearchApp ls;  
    int target, result;  
    ls.arrayInput();  
    cout << "\nArray elements: ";  
    ls.printArray();  
    // Linear Search Section  
    cout << "\nEnter element to search: ";  
    cin >> target;  
    result = ls.search(target);  
  
    if (result != -1)  
    {  
        cout << "Element " << target  
            << " found at index: " << result << endl;  
    }  
    else  
    {  
        cout << "Element " << target  
            << " not found in the array." << endl;  
    }  
}
```

```
return 0;  
}
```

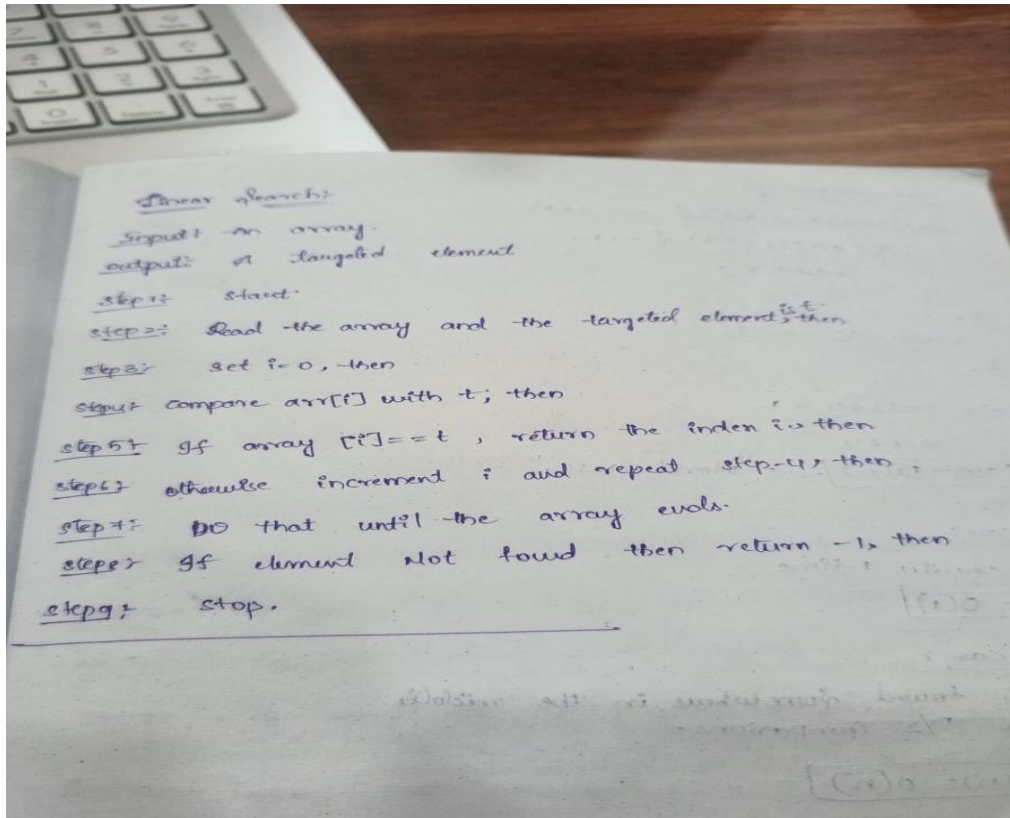
OUTPUT :

```
Output  
Enter number of elements: 5  
Enter the elements: 12  
4  
1  
8  
123  
  
Array elements: 12 4 1 8 123  
  
Enter element to search: 8  
Element 8 found at index: 3  
  
=== Code Execution Successful ===
```

Time complexity :

```
}  
  
int search(int target) {           → T(n)  
    for(int i=0; i<n; i++) {       → (n)  
        if(arr[i]==target) {  
            return i;             → (1)  
        }  
    }  
    return -1;                     → (1)  
}  
  
⇒ Time complexity:  $T(n) = n + 1 + 1 = n + 2$   
⇒  $T(n) = O(n)$ 
```

Binary Algorithm:



Program:

```

#include <iostream>
using namespace std;

```

```

class BinarySearchApp

```

```

{

```

```

public:

```

```

    int arr[100], n;

```

```

    void arrayInput()

```

```

    {

```

```

        cout << "Enter number of elements: ";

```

```

        cin >> n;

```

```
cout << "Enter the elements in sorted order: ";  
for (int i = 0; i < n; i++)  
{  
    cin >> arr[i];  
}  
}
```

```
void printArray()  
{  
    for (int i = 0; i < n; i++)  
    {  
        cout << arr[i] << " ";  
    }  
    cout << endl;  
}
```

// Binary Search Function

```
int search(int target)  
{  
    int low = 0;  
    int high = n - 1;  
    while (low <= high)  
    {  
        int mid = low + (high - low) / 2;  
        if (arr[mid] == target)  
        {  
            return mid;  
        }  
        else if (arr[mid] < target)  
        {
```

```
        low = mid + 1;
    }
    else
    {
        high = mid - 1;
    }
}
return -1;
}
};

int main()
{
    BinarySearchApp bs;
    int target, result;
    bs.arrayInput();
    cout << "\nArray elements: ";
    bs.printArray();
    // Binary Search Section
    cout << "\nEnter element to search: ";
    cin >> target;

    result = bs.search(target);

    if (result != -1)
    {
        cout << "Element " << target
            << " found at index: " << result << endl;
    }
}
```

```
else  
{  
    cout << "Element " << target  
        << " not found in the array." << endl;  
}  
return 0;
```

}Output:

```
Output  
Enter number of elements: 5  
Enter the elements: 12  
4  
1  
8  
123  
  
Array elements: 12 4 1 8 123  
  
Enter element to search: 8  
Element 8 found at index: 3  
  
=== Code Execution Successful ===
```

Time Complexity :

